In Java, il OO è contaminato da alcune parti imperative del C (OO non è puro). In Scala, invece, OO completamente puro

05

Modern, pure and terse object-orientation with Scala

Conciso

Design molto più moderno rispetto a quello di Java

Java è molto 'verbose', mentre Scala è conciso (che aiuta anche la scalabilità: ridotto numero di quantità di codice)

Mirko Viroli

mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Key points of OO-FP integration

Advantages of OO

- scalability/maintainability of code
- encapsulation of data and operations
- small gap in problem vs solution domain
- more productive in-the-large for non-experts

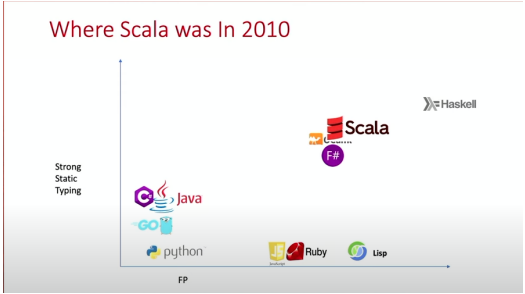
Advantages of FP

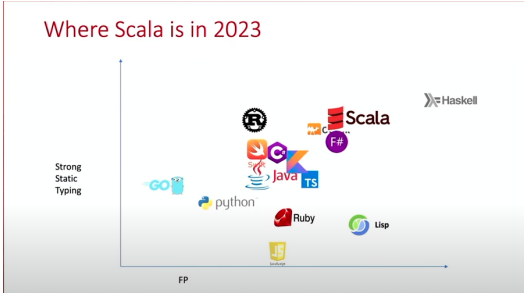
- flexibility and reuse of code
- promote aggressive identification of layers of abstraction
- declarativity gives confidence in correctness, and well suits management of concurrency
- more productive in-the-small for experts

Mixing OO-FP (the direction that Scala pioneered and most fully develops)

- aims at retaining advantages of both worlds
- promote immutable structures, supporting declarative programming
- allow mutable structures and side-effects where needed
- very wide design space, code needs to be carefully crafted and disciplined

The role of Scala in lang ecosystems (credits to M.Oderski)





Andando verso destra si hanno sempre più aspetti funzionali
Andando verso l'alto si hanno più controlli di errore a livello statico (senza eseguire) [Aspetto di Software Engineering]

Scala features revisited

A Scalable language

- Scala is OO: data and operations are packed together, it is also pure
- Scala is functional: functions are first-class, operations with no side-effect are promoted
- Scala is Java-compatible, concise, high-level, statically typed (and sophisticated. . .)

Roots Scala ha preso vari aspetti da altri linguaggi

- syntax from Java/C#/C++, lately moving away from them significantly, to Python
- basic types, libraries and execution model are inherited from Java/JVM
- uniform object model from Smalltalk, then Ruby
- universal nesting from Algol and Simula
- uniform access for methods and fields from Eiffel
- functional features and typing from ML
- advanced function features from Haskell, like type classes
- Actors from Erlang

Scala: more on our progression

Stages

1. purely functional fragment, to stress key FP concepts (done)
2. Scala as an OOP language, pure and modern (today)
3. diving into consequences of OOP-FP mixing (next)

This lesson

- try to stick to the OOP fragment of Scala, which naturally builds on the JVM
- explain elements of modernity (uniformity, flexibility, conciseness)
- leave discussions on how to trade off FP and OOP to next lessons

Outline

1. Basic OO
2. Uniform OO access in Scala
3. Case classes and companion object
4. Control structures

Standard OO

In Scala, non esiste keyword static

Scala as a pure OO language

- by construction, Scala maps to Java, and provides OO mechanisms
- most constructs have a one-to-one mapping
- additional features apparently make Scala an even purer OO lang
- ...others are added to directly implement some basic patterns

Scala OOP concepts (will be detailed in the following)

Un oggetto è un entità che ha uno stato, identità ed offre servizi attraverso delle chiamate

- the `class` concept exists in both languages

Una classe è un template per generare oggetti

- `trait`: basically interfaces/types (but more: they are *mixins*!)

un interfaccia cattura solo il contratto dell'oggetto/classe, cioè intestazione dei suoi metodi (un trait è molto di più di un'interfaccia)

- `object`: a module (a group of static properties), also a singleton

object si può usare sia per definire metodi statici che per definire singleton (in scala, definisco una classe e il suo unico oggetto) (risolve il problema che c'è nei linguaggi OOP come Java, dove vengono usati Metodi Statici cosa non ammessa dal paradigma " everything is an object": è un "tradimento" del paradigma a oggetti puro, perché crea membri che appartengono alla classe e non a un'istanza (un oggetto):)

- `method`: a named function defined into a class (or object)
- `field`: a value/variable defined into a class (or object)

anche in scala è possibile fare side-effect: campi mutabili

- method call/object creation: mostly as expected

A Java-like, OO-style support for counters (Scala 2)

anche in Scala 3

1:1 a quello che avrei scritto in Java

```
// A trait defines a type, it includes a contract and possibly an incomplete class
trait Counter {
  def increment(): Unit // Unit is the equivalent of "void"
  def getValue(): Int
}

// a class with no constructors, i.e., only default constructor
class CounterImpl extends Counter {

  private var value: Int = 0 // a mutable field
```

(viene riunificato extends ed implements)

side-effect in Scala

```
  // public is the default visibility
  // override is a keyword, mandatory for actual overriding
  override def increment(): Unit = {  // return type could be omitted
    this.value = this.value+1 // operator ++ not available...
  }

  override def getValue(): Int = {
    return this.value // return keyword not needed, but possible
  }
}

// a module, with a main
object UseCounter { // note an object could extend a class..
  def main(args: Array[String]) = {   // Array as a standard generic class
    val counter: Counter = new CounterImpl() // "()" typically always optional
    counter.increment()
    counter.increment()
    counter.increment()
    println("Result is "+counter.getValue()) // 3
  }
```

Quando una classe ha solo metodi statici, bisogna usare object in Scala (classe che ha solo metodi statici ed un solo oggetto: un Singleton)

Per rendere immutabile, in Java, bisogna usare final. In Scala, bisogna usare val.

In Java, sarebbe tipo primitivo (suggerito l'uso di Liste al posto di Array, cosa non necessario in Scala). In Scala, viene definito come classe (sarebbe una Collection)

In Scala, se getter (non ho side-effect, solo ritorno valore), la prassi dice che, non si devono usare le tonde

Basic OOP in Scala

Main elements

- most basic OO Java constructs have a 1-to-1 mapping
- advanced aspects (currying, module types, algebraic types, ...) have no clear mapping!
- so far we do not know how to build constructors (see next slide)
- `public` is not a keyword, it is the default (surprisingly... will see why)
- `override` as a keyword (too important to just be an annotation)
- `object` construct an interesting one to deepen later la keyword 'object' viene usata quando ho classe che ha solo metodi statici (singleton)
- interesting use of `println`, where is it defined?
- will see a number of syntactic tricks to make programming more agile

Relating to previous known FP contexts

- a `class` is a template to generate multiple objects: the class is their (super-)type
- an object has an identity, and its fields could be mutable to program a mutable state
- an object itself, turns out to be similar to a module (possibly with state)

⇒ essentially: that's all OOP specifically brings to our previous FP fragment

Equivalent solution in Scala 3, with some idiomatic conventions

NON USARE SCALA COME SE FOSSE JAVA

Non usare le graffe, ma i punti ed indentazione

```
object CountersScala3:
  trait Counter:
    def increment(): Unit
    def value: Int

  class CounterImpl extends Counter:
    private var v: Int = 0
    override def increment(): Unit = v = v + 1
    override def value: Int = v

  @main def tryCounters() =
    val counter: Counter = new CounterImpl  // "()" typically always optional
    counter.increment()
    counter.increment()
    counter.increment()
    println("Result is " + counter.value) // 3
```

Tutto quello che scriviamo, può essere messo in un modulo (non obbligatorio): mettere una classe dentro ad un object ha solo un impatto sul nome assoluto, quando vogliamo accederci da fuori (cosa che in Java è sconsigliato fare, cioè mettere classi innestate (ricordati che object è una classe con un solo oggetto e solo metodi statici)). Fare questa cosa è molto simile da avere un sotto-package: si può fare entrambi.

Se corpo di un metodo ha solo una riga e corta, metterlo subito affianco all'intestazione

Se ho un metodo che fa solo return di qualcosa, si può usare questo stile

Possiamo mettere main in un metodo, annotandolo @main

è opzionale se il costruttore non accetta argomenti

Getter non si mette le tonde, quindi non lo chiamo nemmeno getValue, ma direttamente value

Constructors

I costruttori sono metodi statici della classe

Recall that:

in modern OOP, we tend not to like constructors very much, especially many in the same class, we tend to use (static) factories instead

Primary constructor

è integrato direttamente nella dichiarazione della classe.

I parametri sono definiti subito dopo il nome della classe. Tutto il codice contenuto nel corpo della classe (che non sia un metodo o una variabile) viene eseguito al momento dell'istanziazione.

- Scala has a peculiar, very smooth syntax for it
- already somewhat seen with `case class`es syntax
- arguments are indicated like they were sort of class parameters
- additional code is inserted as "floating" wherever in the class
  - it is highly suggested to add such instructions on top, typically after additional fields and constructors
  - recall Scala is scalable and "nesting-oriented"

Sconsigliato, nelle buone pratiche, avere molti costruttori

Auxiliary constructors

Se hai bisogno di diversi modi per creare un oggetto (overloading dei costruttori), usi i costruttori ausiliari.

- not very much used (consider to use Builders or Factories)
- not a very good idea to use them, per clean code guidelines
- Java-like syntax, with necessary call to `this`

Constructors in Scala: Lamp

I 'class parameters' possono essere preceduti da val (immutabili) o var (mutabili).

```
class Lamp(on: Boolean):  // a class with two constructors

  private var state: Boolean = on
  private val firstUse: java.util.Date = new java.util.Date()

  // floating code is executed as part of the primary constructor
  println("primary constr. of: " + this)

  // auxiliary constructor, necessarily calling the primary one
  def this() =
    this(false)
    println("auxiliary constr. of: "+this)

  def isOn: Boolean = state

  def switchOn(): Unit = state = true

  def switchOff(): Unit = state = false

  override def toString(): String =
    s"Lamp {state} first use {firstUse}; identity {super.toString}"
```

un Costruttore primario ed uno Ausiliario

'on' non diventerà campo, ma serve solo per inizializzare il vero campo

Scala vede tutta la libreria di Java

Per obbligo, gli ausiliari si devono appoggiare al primario

toString gestito come in Java

Se il parametro serve solo a inizializzare altri campi o viene usato solo nel codice che viene eseguito all'avvio (il corpo della classe), il compilatore lo tratta come un semplice parametro del costruttore. Una volta creato l'oggetto, quel valore può essere rimosso dalla memoria perché non serve più. Se invece il parametro viene richiamato all'interno di un metodo (def), Scala deve "tenerlo stretto". Poiché il metodo può essere chiamato in qualsiasi momento del ciclo di vita dell'oggetto, il compilatore trasforma silenziosamente quel parametro in un campo privato dell'oggetto (private field). Inoltre, se uso val/var/private nei parametri della classe, diventa automaticamente un campo

Scala/Java interoperability

Aspects of Scala design

- mostly, Scala code has a simple Java equivalent
- when targeting the JVM, Scala compiler creates Java-compatible byte-code
- actually Scala nicely compiles to JS/Native as well!
- structure of packages is similar in the two languages

Consequence: good interoperability

Codice Java può essere tranquillamente usato in un progetto Scala perché il compilatore Scala ha una dipendenza con compilatore Java e quindi riesce ad eseguire anche codice Java

- Scala classes can smoothly "see and use" Java ones
- Java classes can (not so smoothly) "see and use" Scala ones
- typically, Scala projects in IDEs (like IntelliJ) can have classes of both kinds
  - pay attention to `javacOptions` section in SBT
- (Note: Java-calling-Scala strict interoperability was never a goal, differently to other languages like Kotlin)

Completely equivalent Java implementation

```java
public class JLamp {

    private boolean state;
    private final java.util.Date firstUse = new java.util.Date();

    public JLamp(final boolean on){
        this.state = on;
        System.out.println("primary constr. of "+this);
    }

    public JLamp(){
        this(false);
        System.out.println("auxiliary constr. of "+this);
    }

    public boolean isOn(){
        return state;
    }

    public void switchOn(){
        state = true;
    }

    public void switchOff(){
        state = false;
    }

    @Override
    public String toString() {
        return "Lamp " + state + " first use " + firstUse +
                " identity " + super.toString();
```

Client code: UseLamp

```
@main def tryLamp =
  println("starting..")
  val v = new Lamp(false)
  println(v)
  println(v.isOn) // omitting () at the call site, for getters
  v.switchOn()
  println(v.isOn)

  val v2 = new Lamp(true)
  println(v2.isOn)
  val v3 = new Lamp  // omitting () at the call site, for constructors
  println(v3.isOn)

  val v4 = new JLamp  // Java interoperability
  println(v4.isOn)
```

Class parameters vs fields

Di solito, i campi in Scala hanno underscore davanti (viene fatto in particolare quando si ha un getter, dato che i getter hanno stesso nome del campo, così il campo ha underscore)

il comportamento dei parametri della classe è dichiarativo significa che tu, come programmatore, descrivi "cosa" vuoi ottenere (la logica della classe) e lasci che sia il compilatore a decidere "come" implementarlo a basso livello (se creare un campo in memoria o meno).

Class parameters vs fields

- different roles in principles (constructor arguments vs state)...
- ...often "the same thing", especially for conciseness (DRY), and most typically in FP-like settings
- in principle, a constructor is not very much different to a static method...
- in Scala there are ways of making class parameters "also fields"

How to automatically make a class parameter also a field? It's somewhat declarative:

1. if the parameter comes with `val`/`var` definition (possibly `private`, if needed)
2. if the parameter is used by some method body, in which case it is assumed `val` and `private`
3. in a `case class` the parameter is treated as a public immutable field

Why all this?

- essentially: whether parameters need not be fields is just an optimisation aspect
- classes are mere implementations, need ways for creating them smoothly!

Idiomatic Scala 3 solution: IdiomaticLamp

```scala
// use significant indentation!
object IdiomaticLamp:  // not required, but often wrap into an object module

  class Lamp2(private var _state: Boolean):  // undescore for private vars
    def isOn: Boolean = _state // almost never use curly brackets
    def on() = _state = true   // Unit return type as sort of default
    def off() = _state = false
    override def toString = s"Lamp $_state identity ${super.toString}"

  @main def useLamp() =   // entry-point main by annotation
    val v = Lamp2(false) // 'new' is optional: won't use it
    println(v)
    println(v.isOn)
    v.on()
    println(v.isOn)

  @main def usePoint() =   // entry-point main by annotation
    // the degenerate case of a one-line class
    class Point(val x: Int, val y:Int)
    val p = Point(10, 20)
    println(s"$p.x, $p.y")
```

"degenerate" si riferisce a una situazione in cui un'entità è talmente semplificata da ridursi ai suoi elementi minimi, perdendo quasi la sua forma abituale.

In Java, ho bisogno del costrutto record, cosa che in Scala è già tutto pronto

Inheritance works as expected!

Funziona esattamente come in Java

Se non mettiamo nulla nelle definizioni, sono public di default

```
object CountersAndInheritance:

  trait Counter:
    def value: Int
    def increment(): Unit

  class CounterImpl(protected var _value: Int) extends Counter:
    override def value: Int = _value
    override def increment(): Unit = _value = _value + 1

  trait DecrementCounter extends Counter:
    def decrement(): Unit

  class DecrementCounterImpl() extends CounterImpl(0) with DecrementCounter:
    override def decrement(): Unit = _value = _value - 1

  @main def main() =
    val c: Counter = new CounterImpl(0)
    c.increment()
    c.increment()
    println(c.value)
    val c2: DecrementCounter = new DecrementCounterImpl
    c2.decrement()
    println(c2.value)
```

trait sono quasi classi astratte (quindi, in Scala, abbiamo quasi ereditarietà multipla)

Ordine conta: vedremo extends Classe with Traits

In Java, per estendere più cose si usava extends e implements con la virgola. In Scala, la prima cosa che si estende viene messa dopo extends (quando si usa extends, bisogna specificare il class parameter, se li usa: in Java, si scriveva nel costruttore un super(...)) e poi si usa il with ripetuto per ogni classe da estendere. In Scala, come in Java, non si ha ereditarietà multipla, ma singola e si possono estendere multiple interfacce

Inheritance as expected

Notes

- "extend one class, implements many traits" as in Java
- keyword changes: first is `extends`, others are `with`
- Java's `super` instruction comes right `extends` clause
- `protected` as expected gives visibility to subclasses
- other details to come. . .

Objects as singletons

Classe con soli metodi statici e con un solo oggetto (quindi viene implementato il pattern singleton)

The problem

In OOP you really want to use static methods for two reasons

1. have a class with helper functions only, i.e., a module
2. have a single "object" attached to a type, i.e., a singleton

si riferisce all'idea di usare la classe stessa come se fosse l'unica "istanza" esistente

⇒ problem: static calls have a different "polymorphic" semantics

The Scala solution, an `object`

oggetto di un unica classe

- it serves both purposes: a module in FP is similar to a class-less object in OOP

  'senza classe'
- its methods/fields are accessed directly on the object's global name
- still an object: can extend a class or implement a trait, and be treated like any other object
- this way, Singleton can no longer be seen as an anti-pattern, since your access to a singleton can actually hide an injected dependency

Objects as singletons

In Java, le classi con metodi statici non aderiscono ad un contratto (non possono implementare un'interfaccia). In Scala, è possibile.

```
object Singleton extends App :

  trait SequenceGenerator:
    def get: Int

  class SequenceFromSupplier(f: () => Int) extends SequenceGenerator:
    override def get: Int = f()

  val s: SequenceGenerator = new SequenceFromSupplier(() => 1)
  println(s.get)
  println(s.get)

  object Zeros extends SequenceFromSupplier(() => 0)

  object GlobalCounter extends SequenceGenerator:
    private var n: Int = 0
    override def get: Int =
      n = n + 1
      n - 1

  val s2: SequenceGenerator = GlobalCounter
  println(s2.get)
  println(s2.get)
  println(GlobalCounter.get)
```

Quando usiamo Singleton in OO (in Java, C#, etc..), se dobbiamo modificare l'implementazione, devo andare in tutti i punti che uso il Singleton e cambiare, cosa che non accade in Scala (la flessibilità del 'object' ci permette di applicare modifiche in agilità)

In Java, esiste una dualità:
- Ci sono gli Oggetti (OO).
- Ci sono i Membri Statici (che vivono fuori dal mondo degli oggetti, stile di programmazione C-like).

In Scala, questa dualità scompare. Quello che in Java è statico, in Scala diventa un Singleton Object (object).
- Vantaggio: Un object in Scala può implementare interfacce (Trait), ereditare da altre classi e passare se stesso come argomento a funzioni. In Java, non puoi passare "la parte statica di una classe" come parametro.

Scala OOP: takeaway so far

Phylosophy

Can do standard OOP in a modern way

Features

- `trait` as interface/abstract class
- `class` as usual
- `val`/`var` for fields
- primary constructor via class parameters and body
- `object` as module (of static methods) or pure-OOP singleton
- Scala 3 compact syntax to clarify the design

Guidelines

- just class parameters
- names without parens for getters, use underscore for private vars
- shortest method bodies, hopefully just one expression

What's next? Scala's addresses several Java weaknesses

Weakness of Java (C#... mainstream OOP)

- primitive types and operators, vs OOP
- arrays vs classes
- fields vs getters and setters
- static vs instance members
- constructors vs factories
- `equals`, `hashCode`, `toString`

Outline

1. Basic OO
2. Uniform OO access in Scala
3. Case classes and companion object
4. Control structures

Scala paradigm philosophy

Dobbiamo vedere Scala come un linguaggio di superficie sopra Java per risolvere i problemi di Java

Scala is actually, first-of-all, pure OO!!

- all primitive types (numbers, booleans,..) are actually seen as objects (possibly optimised to values)
- all types (also Java classes) have a common, OO-ancestor class
- all operations are just methods (mathematic, field access, function call,. . .)
- special syntax recovers non-OO aspects (e.g., function types/values)
- functions are actually objects of `scala.FunctionN`), called with method (`apply(...)`)
- special syntax promote terseness and may help clarify your design

Will those be features or tricks?

- non-experienced Scala programmers tend to see them as tricks
- experienced ones just happily leverage them all!
- train yourself to combine the two views as needed
  - enjoy the coherence of the "surface OO language"
  - enjoy the sugar and get acquainted with it, it makes programming very fun

Basic Scala types: everything is an object, truly

La gerarchia dei tipi di Java: esistono due mondi separati:
- Il mondo degli Oggetti: Tutto ciò che eredita da java.lang.Object. Qui trovi le classi che definisci tu, le stringhe, le collezioni (ArrayList, HashMap), ecc.
- Il mondo dei Primitivi: Sono 8 tipi speciali (int, boolean, char, double, ecc.). Non sono oggetti. Non hanno metodi, non puoi chiamare .toString () su un int, e non ereditano da Object.

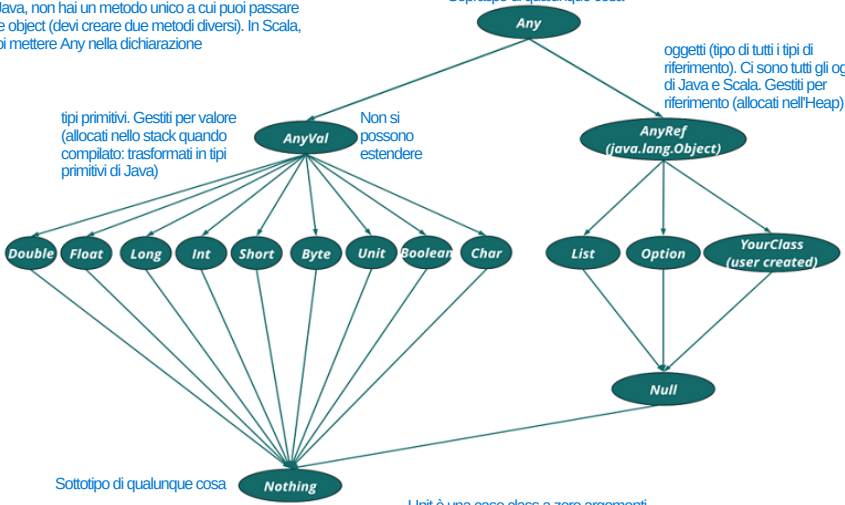
In Java, non hai un metodo unico a cui puoi passare int e object (devi creare due metodi diversi). In Scala, puoi mettere Any nella dichiarazione

Sopratipo di qualunque cosa

oggetti (tipo di tutti i tipi di riferimento). Ci sono tutti gli oggetti di Java e Scala. Gestiti per riferimento (allocati nell'Heap)

tipi primitivi. Gestiti per valore (allocati nello stack quando compilato: trasformati in tipi primitivi di Java)

Non si possono estendere



Sottotipo di qualunque cosa

Unit è una case class a zero argomenti
In Java, il null è una keyword, non un tipo come in Scala

Dire che la gerarchia dei tipi di Scala è un lattice (o reticolo) significa che è una struttura perfettamente chiusa sia verso l'alto che verso il basso.

In Java, la gerarchia è un albero che punta verso l'alto a Object, ma "sfuma" verso il basso (non esiste un tipo che sia figlio di tutti). In Scala, invece, ogni coppia di tipi ha sempre un antenato comune più vicino (join) e un discendente comune più vicino (meet).

Basic Scala types

Notes

- `scala.Any` is the root of all (values and references)
- `scala.AnyVal` is the root of values (including `Unit`) – may be optimised
- `scala.AnyRef` is the root of references, and is aliased to `java.lang.Object`
- `java.lang.String` is the type of Scala strings as well (`scala.String`)
- `scala.Null` is the type of `null`, subtype of all references
- `scala.Nothing` is the type of `throw` expressions and `???`, subtype of all

Try in REPL

- `val v = ()=>{???}`
- `:type ()=>{???}`
- `val v = ()=>null`
- `val v = ()=>throw new Exception()`
- `val v = ()=>if (true) then 10 else ???`
- `val v = ()=>if (true) then 10 else true`

Operators are method calls

```scala
@main def typesExample() =  // a top-level main method

  val a: AnyVal = 1.2
  val b: Any = Object()
  val c: Null = null
  val e: java.util.Date = null // null is a Java thing..
  // val n: Nothing = throw new Error()
  // val iterable: Nothing = ??? // scala.NotImplementedError

  val i = 10
  val j = 10 + 20
  val k = i.+(20) // binary arguments are just method calls
```

Per massimizzare le performance, Scala userà un int primitivo di Java, ma ti permetterà di trattarlo come un oggetto.

in Scala non c'è differenza sintattica tra un'operazione tra primitivi e una chiamata a un metodo su un oggetto.

In realtà, anche questo è un metodo! In Scala, l'operatore + è un metodo definito nella classe Int. Scrivere 10 + 20 è solo zucchero sintattico (una forma più leggibile) per scrivere 10.+(20).

In Scala, qualsiasi metodo che accetta un solo argomento può essere usato come un operatore (senza punto e senza parentesi).

Functions are objects, function call is apply invocation

In Scala, tutto il layer di FP è, in realtà, sintassi sopra il mondo OO

Classe

```
val g: Int => Int = f // Function1[Int,Int] an alias for (Int)=>(Int)
println(f) // <function1>
println(f(1)) // 2
```

Stiamo chiamando su quel "Any" il metodo Apply

```
println(f.apply(1)) // 2, calling a function is invocation of apply
```

```
println((f compose g compose f)(1)) // 4
```

funzionalità compose sulle funzioni, che in realtà sarebbero chiamate a metodo sulla funzione a sinistra e come argomento la parte di destra

```
println((f.compose(g.compose(f)))(1)) // 4

val f2: Function2[Int, Int, Int] = _ + _ //same as val f2: (Int, Int) =>
  Int
println(f2) // <function2>
println(f2(1, 2)) // 3
```

Uniform access: details

Method names

- no need to make it different than math/logical operators...
- since math/logical operators *are* methods (extension or standard methods, it is not so important)
- expressions with methods may appear as "(math-like) expressions"

Methods vs operators

- each unary method can be called with infix notation
- operators ending with "`:`" are right-associative (used for left-to-right constructions, e.g., lists)

Getters/setters and indexers

- methods without arguments can be called without parenthesis, resembling field access (guidelines say to do it only if they have no side effect)
- special method `XYZ_=` is a setter, called with reassignment syntax
- methods `apply`/`update` are used to support array-like indexers
  - arguably, more streamlined than C# properties/indexers

Showcasing a bit of uniform access: methods vs operators

In Scala, non esistono operatori built-in fissi. Anche il + degli interi è solo un metodo. Questo ti permette di creare delle "DSL" (Domain Specific Languages), ovvero scrivere codice che sembra un linguaggio specifico per il tuo problema (es. operazioni tra matrici o query database) usando simboli matematici invece di lunghi nomi di metodi.

```scala
class A:
  var field = 10
  def meth(s: String):String = s + 10
  def ***(s: String):String = s + 10   // just a method called ***
  def prop: Int = field                // a getter, note lack of "()"
  def prop_=(i: Int): Unit = field=i   // corresponding setter
  def apply(index: Int): String = "indexer-get"
  def update(index: Int, s: String): Unit = println("indexer-set")
  def +:(i: Int):A = { field = field + i; this } // +: gives right-associativity
```

```scala
val obj = new A
val str = obj.meth("ciao") // four equivalent versions..
val str2 = obj meth "ciao"
val str3 = obj *** "ciao"
val str4 = obj.***("ciao")
println(str3)
```

Significa che in Scala non esiste una categoria speciale di "operatori" integrati nel linguaggio (come il + o il * in Java o C). Quelli che sembrano operatori sono in realtà semplici metodi con nomi simbolici. Questa è la massima espressione dell'Uniform Access Principle: il linguaggio non fa distinzione tra un'operazione matematica e la chiamata a una funzione.

Showcasing a bit of uniform access: getters/setters vs field access

Quando scrivi var field = 10 in una classe Scala, il compilatore non crea semplicemente una variabile pubblica. Crea tre cose:
- Un campo privato (nascosto): private int field.
- Un metodo getter pubblico: def field: Int.
- Un metodo setter pubblico: def field_=(i: Int): Unit.

Quando io accedo ad un campo di un oggetto, sto accedendo al campo o sto chiamando un metodo? In Scala, non lo so perchè compila sia se è una val pubblica dentro la classe di un oggetto sia se è un metodo pubblico senza parentesi tonde (sono la stessa cosa)

```
class A:
  var field = 10
  def meth(s: String):String = s + 10
  def ***(s: String):String = s + 10   // just a method called ***
  def prop: Int = field                // a getter, note lack of "()"
  def prop_=(i: Int): Unit = field=i   // corresponding setter
  def apply(index: Int): String = "indexer-get"
  def update(index: Int, s: String): Unit = println("indexer-set")
  def +:(i: Int):A = { field = field + i; this } // +: gives right-associativity
```

```
println(obj.prop) // 10
obj.prop = 1
println(obj.prop) // 1
```

Perché non viola l'incapsulamento avere val/var pubbliche? In Java, impostiamo i campi come private per rendere facilmente manutenibile il codice in caso di modifiche future (esempio: se il campo lo metto public e nel futuro voglio aggiungere dei controlli, devo andare a modificare tutte le parti del codice che usano direttamente il campo e sostituirlo con il getter con i controlli). In Scala, un campo pubblico non è mai davvero un "campo nudo" come in Java: non c'è questo problema perchè il codice è facilmente manutenibile per definizione (se voglio cambiare il codice, basta aggiungere manualmente prop e prop_ come getter e setter del campo senza modificare il resto del codice). In Java, public int x è un accesso diretto alla memoria. In Scala, val x: Int è un'astrazione che il compilatore trasforma automaticamente in un metodo.

Showcasing a bit of uniform access: indexers, i.e., array-like get/set

```
class A:
  var field = 10
  def meth(s: String):String = s + 10
  def ***(s: String):String = s + 10   // just a method called ***
  def prop: Int = field                // a getter, note lack of "()"
  def prop_=(i: Int): Unit = field=i   // corresponding setter
  def apply(index: Int): String = "indexer-get"
  def update(index: Int, s: String): Unit = println("indexer-set")
  def +:(i: Int):A = { field = field + i; this } // +: gives right-associativity
```

è equivalente ad aver fatto un update (che deve essere implementato)

```
println(obj(10)) // indexer-get, java's obj[10]
obj(11) = "casa" // indexer-set, java's obj[11]="casa"
obj.update(11, "casa") // same as above
```

Se obj è un Any, sarebbe obj.apply(10). L'apply è descritto manualmente nella classe A

Serve per avere una sintassi tipo quella degli array per accedere ad un contenuto di un array o in lettura o in scrittura. Dato che in Scala non ci sono gli Array, che sono un tipo di dato come gli altri, non abbiamo questa sintassi (esempio, lettura del decimo elemento dell'array). Quindi, usiamo gli indexers per usare la stessa sintassi degli array (uso del metodo apply) sulle liste ed altre collezioni, usando le tonde al posto delle []

Showcasing a bit of uniform access: right-associative operators

```
class A:
  var field = 10
  def meth(s: String):String = s + 10
  def ***(s: String):String = s + 10    // just a method called ***
  def prop: Int = field                 // a getter, note lack of "()"
  def prop_=(i: Int): Unit = field=i    // corresponding setter
  def apply(index: Int): String = "indexer-get"
  def update(index: Int, s: String): Unit = println("indexer-set")
  def +:(i: Int):A = { field = field + i; this } // +: gives right-associativity
```

```
10 +: 20 +: 30 +: obj //obj.+:(30).+:(20).+:(10) --> 61
println(obj.prop) // 61
```

Quando il nome del metodo finisce con ":" allora viene applicata l'associatività a destra

Il compilatore vede 30 +: obj. Poiché finisce con :, capisce che il metodo appartiene a obj. Esegue: obj.+:(30).
Il risultato di questa operazione è un nuovo oggetto (o lo stesso obj modificato, come nel tuo codice alla riga 9).
Ora l'espressione è diventata: 10 +: 20 +: (risultato_precedente).
Applica di nuovo la regola: (risultato_precedente).+:(20).
Infine: (ultimo_risultato).+:(10).

Some deep consequences

Quindi, qualunque operazione in Scala è, di fatto, una chiamata di metodo fatta su qualcosa che è un oggetto

Key fact 1: truly, every runtime entity could be considered as an object!

- no distinction: primitive types vs objects vs functions
- no wrappers confusion as in Java
- can apply generics to any type (`Array[Int]`, `Array[Lamp]`, ..)
- smart syntax help you not become bored of method calls
- still, Scala can smartly compile `AnyVals` into primitive Java types (same with arrays)

Key fact 2: truly, every operation could be considered as a method call!

- applying a function, a math operator, accessing/setting a field (, creating an object)
- no true distinction between fields and methods at the call site
- exposing fields is the same as exposing a getter (and/or setter)

⇒ it does not violate encapsulation
⇒ it is managed with good performance under-the-hood
⇒ similar, but more profound to C# properties
⇒ motivates the reason why `public` is a default and not a keyword at all

Scala combines fields and methods

Differences between fields and 0-ary methods

- `def`: the 0-ary method executes its definition each time is called
- `val`: the immutable field is evaluated at construction time
- `var`: the mutable field also has a setter

Technical consequence

- there's no "usage" differences between 0-ary `def` and `val`/var (che ha in più solo il fatto che è mutabile)
- a contract (`trait`) can include `def` or `val` (or `var`) → perchè alla fine sono solo modi diversi per chiamare un oggetto (quando chiamo una val e come se chiamassi una def; quando chiamo una var per modificare qualcosa e come se chiamassi una def)
- a `val` can be implemented only with a `val`
- a `def` can be implementation either with `def` or `val`
- using `def` in contracts is preferrable, since more abstract
- use `val` only to declare that the operation always returns the same
- consider avoiding "`()`" in 0-ary `def` with no side-effect

Methoodlogical consequence

- no problem if we expose fields (`val`/`var`) to a client class
- later on we can trasparently revert them to `def`(s) to encapsulate new behaviour (OCP)

Uniform access and encapsulation, pt 1: using class parameters

```scala
// A person with immutable name/surname
// mutable married flag, and toString
trait Person:
  def name: String
  def surname: String
  def married: Boolean
  def married_=(state: Boolean): Unit
  // a template method
  override def toString(): String = s"$name $surname $married"

// easiest implementation
// with overriding class parameters becoming fields
class PersonImpl1(override val name: String,
                  override val surname: String,
                  override var married: Boolean) extends Person
```

(non ha setter, quindi immutabile)

Getter su name (immutabile)

Getter su surname (immutabile)

Getter e Setter su married (mutabile)

che aderisce completamente al contratto

(implementati i field tramite val e var)

Uniform access and encapsulation, pt 2: overriding some methods

Qua, solo married override con var e vengono definiti esplicitamente name e surname con metodi (al posto di prima che implementavo con val)

```
// surname as a constant, name with overridden getter
class PersonImpl2(_name: String,
                  override var married: Boolean) extends Person:
  override def name =
    require(_name != null) // may throw IllegalArgumentException
    _name
  override val surname = "?"
```

(sarebbe un if che se la condizione non è vera, lancia un'eccezione

val può essere usata per implementare una def

Uniform access and encapsulation, pt 3: overriding a var with methods

Se in Java è sconsigliato fare campi pubblici, in Scala non lo è: non stiamo rompendo l'incapsulamento perchè la decisione è rivedibile dopo senza modificare tutto il codice nel programma (dato che i field val vengono esposti all'esterno tramite metodi implementati implicitamente: incapsulamento nativo con esposizione esterna sempre tramite getter/setter anche se non scritto esplicitamente)

```
// overridden getters and setters of married
class PersonImpl3(override val name: String,
                  override val surname: String,
                  private var _married: Boolean) extends Person:
  override def married =
    println("getting married property")
    _married
  override def married_=(m: Boolean) =
    require (_married != true || m != false)
    _married = m
```

(al married non è stato fatto l'override con class param, ma tramite definizione del getter e setter con metodo)

Uniform access and automatic encapsulation

Tutti i servizi offerti da un modulo dovrebbero essere disponibili attraverso una notazione uniforme, indipendentemente dal fatto che siano implementati tramite memorizzazione (un campo/variabile) o tramite calcolo (un metodo/funzione). La sintassi di accesso per ottenere un dato deve essere la stessa, a prescindere da "come" quel dato venga generato internamente.

```
// Note uniform access
@main def usePerson() =
  val p: Person = PersonImpl1("mario", "rossi", false)
  println(p)
  val p2: Person = PersonImpl2("mario", false)
  println(p2)
  val p3: Person = PersonImpl3("mario", "rossi", false)
  println(p3)
  p3.married = true
  println(p3)
```

Uniform access: takeaway

Phylosophy

Everything is a method call to an object!

Features

- `Any` and `Nothing` as extremes of the type lattice
- Hence, no distinction betweem primitive/non-primitive, it's just syntax
- Function types/values/calls are syntax for `FunctionX/apply`
- Tuples are syntax for `Tup2, Tup3, Tup4`
- Names can have/be symbols (useful for math-like and DSLs)
- Binary operators are just 1-arg methods
- Constructors are called with method syntax
- Reading/storing fields has same syntax of calling getters/setters
- Indexing through `apply/update`
- `def/val/var` can mutually override, in a principled way

Outline

1. Basic OO
2. Uniform OO access in Scala
3. Case classes and companion object
4. Control structures

Objects equality/hashing/toString in Scala

In Java, "==" controlla se i riferimenti ai due oggetti sono uguali (da usare per tipi primitivi, non tra oggetti); se voglio definire un modo per dire che due oggetti di una classe sono uguali devo fare override di equals

Class `scala.Any` methods

- "`final def ==(that: Any): Boolean`", it calls internal `equals`
- "`final def !=(that: Any): Boolean`", it negates `==`
- "`def equals(that: Any): Boolean`", the one to override if needed
- "`def ##: Int`", it calls `hashCode`
- "`def hashCode: Int`", the one to override if needed
- "`def toString: String`", to override
- method `eq` for reference equality is in `AnyRef`

In Scala, si usa SEMPRE "==" sia per tipi primitivi (che rimane uguale a Java) sia per tipi oggetto (che sarebbe la equals in Java, che come in Java deve essere programmata)

Result

- `==` to be used for object/value equality (dealing with `null`), `eq` for reference equality
- override `hashCode`, `equals` and `toString` as needed
- you better use IDEs support if not expert
- `case classes` already well implement them, based on class parameters

equals: Controlla che i class param siano uguali tra oggetti

Quando si mette case su una classe, il compilatore aggiunge automaticamente alla classe la hashCode, equals e toString ed altre cose (già implementate in un modo che vadano bene: se non vanno bene, fare override)

Objects equality – trickier than Java

```scala
object Equality extends App:

  class Pair[A,B](val x: A, val y: B)

  class PairWithEq[A,B](val x: A, val y: B):
    // IntelliJ generation of code, with few adjustments
    def canEqual(other: Any): Boolean = other.isInstanceOf[PairWithEq[_,_]]

    override def equals(other: Any): Boolean = other match
      case that: PairWithEq[A,B] =>
        (that canEqual this) && x == that.x && y == that.y
      case _ => false

    override def hashCode(): Int =
      Seq(x, y).map(_.hashCode()).foldLeft(0)((a, b) => 31 * a + b)

  case class CasePair[A,B](x: A, y: B)

  println(new Pair(10,20) == new Pair(10,20))  // false
  println(new PairWithEq(10,20) == new PairWithEq(10,20)) // true
  println(CasePair(10,20) == CasePair(10,20)) // true.. don't need 'new'

  val p = new Pair(10,20)
  println(p eq p) // true.. is reference equality!
```

Case classes

Simile a record che si ha in Java (ma più potenti in Scala)

Motivations del suo utilizzo

Fornire una sintassi

~~To provide a syntax~~ to define a ~~record-like class~~ classe simile a record, where construction directly reflects internal structure, and ~~hence~~ quindi automatically defines also matching, equality, string representation and so on

Most specifically, a case class automatically includes

- a public immutable field for each class argument
- `toString`, `equals` and `hashCode` based on class arguments
- `unapply` to support match-case, into the corresponding "companion object" (see next slides)
- methods to clone into a modified version of the object

Mentre apply serve a "costruire" l'oggetto dai pezzi, unapply serve a "smontarlo". Questo permette di usare le case class nel match (i metodi apply ed unapply sono nel companion object)

What about method `apply` as a factory in the "companion object"?

- in Scala 3 this is always automatically added, unless the companion object is explicitly programmed

apply and unapply

apply

- an `object O`'s method `apply` has similar goal of `FunctionX.apply`
- it allows to use notation `O(...)` in place of `O.apply(...)`
- this is typically used to create nice static factories of the corresponding `class O`
- it could also be used into a class, in which case you enable syntax `o(...)` for objects
- note it well supports DIP/OCP (can switch from constructor to factory as needed)

unapply, called extractor

- an object `O`'s method `unapply` has opposite goal, i.e., destructuring an object
- this method is called automatically when performing matches against a "`case O(...)`"
- it can return either `Boolean`, `Option[X]`, or `Option[(X1,..,Xn])`
  - class `scala.Option` is similar to the one we already presented, and is automatically imported
  - use `Some(...)` and `None` to create full/empty optionals
- it is suggested to write `unapply` only in combination with `apply`

Factories hiding implementations

Perchè preferire factory a costruttori? Perchè con la new la creazione di un oggetto dipende dall'implementazione specifica di quella classe e se voglio modificarla devo modificare tutto il codice che la utilizza; se utilizzo le factory invece sono più facilitato nelle successive modifiche dato che devo solo modificare la factory e non tutto il resto del codice (ciò mi permette di usare dei sopratipi come tipo di ritorno).

```
object Apply:

  // another, important application: factories hiding implementation
  trait Person:
    def name: String
    def surname: String

  object Person: // the companion object of trait/class Person
    def apply(name: String, surname: String): Person =
      PersonImpl(name,surname)
    private class PersonImpl(override val name: String,
                             override val surname:String) extends Person:
      assert(name != null && surname != null)

  @main def showcase =
    // Showcasing apply
    val p: Person = Person("mario", "rossi")  // Hiding PersonImpl
    println(s"$p, $p.name, $p.surname")
```

Definisce metodi statici di quella classe/trait (nella parte della classe e trait si ha roba non statica)

In Scala, un Companion Object è un object che ha lo stesso identico nome di una class o di un trait definiti nello stesso file sorgente.

il companion object vede ciò che c'è dentro trait/classe

Tra le funzionalità statiche che si possono mettere, c'è l'apply che sarebbe la factory standard associata a trait/class (bisogna sempre preferire factory a costruttori per creazione oggetti). Apply è di fatto una factory statica su Person.

implementazione privata dentro il companion object (nasconde implementazione dentro l'object)

Qua usiamo l'apply (cioè Person.apply("mario","rossi"))

Factories hiding implementations, with matching

- Some(name, age): Il match ha successo. I valori vengono "estratti" e assegnati alle variabili name e age nel tuo case.
- None: Il match fallisce. Scala ignora quel case e passa a quello successivo.

```
object ApplyUnapply extends App:

  // another, important application: factories hiding implementation
  trait Person:
    def name: String
    def surname: String

  object Person: // the companion object of trait/class Person
    def apply(name: String, surname: String): Person =
      PersonImpl(name,surname)
    def unapply(p: Person): Option[(String, String)] =
      Some(p.name, p.surname)
    private class PersonImpl(override val name: String,
                             override val surname:String) extends Person:
      assert(name != null && surname != null)

  // Showcasing apply
  val p: Person = Person("mario", "rossi")  // Hiding PersonImpl
  println(s"$p, $p.name, $p.surname")
  // Showcasing unapply
  println(p match {case Person(n, s) => n+s}) // mariorossi
  println(Person.unapply(p).map((n, s) => n+s).get) // roughly equivalent formulation
```

Ritorna Opzionale pieno

unapply implementa il case matching (cioè se vogliamo che i nostri oggetti siano usabili in match case, bisogna implementare unapply nel companion object oppure usare case che lo implementa automaticamente)

su Case Person si chiama l'unapply, gli si passa il p, lui lo disassembla nelle sue due componenti (cioè in un Opzionale Pieno, ma se vuoto vuol dire che il match non è passato e si va al successivo: si usa Option perchè il match potrebbe fallire e Option mi dice se è fallito o no)

On pattern matching

A key mechanism of Scala and FP

- provides great abstraction to write algorithms
- it is much easier to read
- it is much easier to statically intercept bugs
- can be used a bit also in variable declaration

Pattern matching in Scala

Can match against:

- indication of a type (improvement over Java's `instanceof`)
- a structure, because of an extractor, hence a case class
- can add a further condition
- can capture a structure/substructure into a variable
- can use underscore to perform partial matching
- can use backquotes (`` `n` ``) to mention the content of a variable in scope (per considerare il match con una variabile già dichiarata)
- can use or-pattern
- the default case makes your pattern a total function

Extractors and case-matching: translation pattern

Pattern Matching richiede unapply, non case class

```scala
class Pair[A,B](val x: A, val y: B)
object Pair:
  def apply[A,B](x: A, y: B) = new Pair(x,y)
  def unapply[A,B](p: Pair[A,B]): Option[(A,B)] = Some((p.x,p.y))

class MyPair(override val x: Double, override val y: Double) extends Pair(x,y)

def match1(p: Pair[Double,Double]): String = p match
  case p: MyPair => "MYPAIR"  // matching by type: a safe 'instanceof'
  case Pair(_,0) => "X-axys"  // using unbounded parameters
  case Pair(x,y) if x==y => "X=Y"  // conditional case
  case _ => "?"

// Rough translation into equivalent code
def match2(p: Pair[Double,Double]): String =
  val unapplyResult = Pair.unapply(p)
  if p.isInstanceOf[MyPair] then "MYPAIR"
  else if unapplyResult.collect{case (_, 0) => true}.isDefined then "X-axys"
  else if unapplyResult.filter{case (x, y) => x==y}.isDefined then "X=Y"
  else "?"

println(match1(Pair(10,0))) // X-axys
println(match1(Pair(0,2))) // ?
println(match1(new MyPair(0,2))) // MyPair
```

Matches with variables

applicazione del Pattern Matching nelle dichiarazioni di variabili.

In Scala, il meccanismo di "smontaggio" (unapply) che abbiamo visto nel match-case non è limitato solo ai blocchi match, ma può essere usato ovunque tu definisca una variabile con val o var.

```
// Matches in variable declaration
val t = (10, 20)
val (x, y) = t // same as x = t.x, y = t.y
val p = Pair(10, 15)
val Pair(z,w) = p // Exception if it does not match
val p2 = Pair(10, Pair(20, 30))
val Pair(_, Pair(a, b)) = p2 // a = 20, b = 30
```

se unapply dovesse restituire None (perché magari hai un estrattore filtrato come quello OfLength di prima), il programma lancerebbe un'eccezione (MatchError), perché in una dichiarazione val il match deve avere successo.

All matches

(non possiamo usare delle variabili definite prima di => su quella riga perchè se passa il primo caso, ad esempio, e uso y del secondo caso avrei un errore perchè non definito nel primo caso)

```
object OfLength: // extracting distance to 0 when greater than 1
  def unapply(p: Pair[Double,Double]): Option[Double] =
    Some(Math.sqrt(p.x*p.x+p.y*p.y)) filter (_>1)
val z = 2
def process(p: Pair[Double,Pair[Double,Double]]): String = p match
  case Pair(1, Pair(1, 1)) => "A"     // matching a structure, defined
  case Pair(2, Pair(0, _)) => "B"     // matching a structure, partially specified
  case Pair(3, null) | Pair (4, null) => "C" // or
  case Pair(5, Pair(x, y)) if x == y => "D"  // conditional case
  case Pair(6, pair @ Pair(_,_)) if pair.y == 1  => "E" // capturing variable
  case Pair(7, Pair(‘z‘,‘z‘)) => "F" + z // logical binding of ‘z‘
  case Pair(8, OfLength(n)) => "G" + n  // usign a custom extractor
  case Pair(9, _: MyPair) => "H"  // usign a type check
  case _ => "?" // default case
```

Prima calcola la lunghezza del vettore e se il vettore ha lunghezza > 1 fa match e "n" è della lunghezza calcolata

Il pattern matching funziona anche sui null

dove z = 2 (quella fuori dal match case). Se non lo mettiamo, lo z esterno viene coperto e crea un nuovo z per quella riga

fa match solo se secondo argomento è di quel tipo

pair è una variabile che memorizza Pair(_,_) da usare dopo nella cond

Quando abbiamo pattern innestato, ci piacerebbe dire qualcosa nella condizione a destra (nel primo matching e come se non ci fosse la parte "pair @" perchè esso non influenza il primo match e viene usato nella condizione se passa il primo match)

la condizione booleana vede anche le variabili nello scope (da riprendere con backtick)

All matches: results

```
object OfLength: // extracting distance to 0 when greater than 1
  def unapply(p: Pair[Double,Double]): Option[Double] =
    Some(Math.sqrt(p.x*p.x+p.y*p.y)) filter (_>1)

val z = 2
def process(p: Pair[Double,Pair[Double,Double]]): String = p match
  case Pair(1, Pair(1, 1)) => "A"    // matching a structure, defined
  case Pair(2, Pair(0, _)) => "B"    // matching a structure, partially specified
  case Pair(3, null) | Pair (4, null) => "C" // or
  case Pair(5, Pair(x, y)) if x == y => "D"  // conditional case
  case Pair(6, pair @ Pair(_,_)) if pair.y == 1  => "E" // capturing variable
  case Pair(7, Pair(`z`,`z`)) => "F" + z // logical binding of `z`
  case Pair(8, OfLength(n)) => "G" + n  // usign a custom extractor
  case Pair(9, _: MyPair) => "H"  // usign a type check
  case _ => "?" // default case

println(process(Pair(1,Pair(1,1)))) // A
println(process(Pair(2,Pair(0,0)))) // B
println(process(Pair(3,null))) // C
println(process(Pair(5,Pair(1,1)))) // D
println(process(Pair(6,Pair(1,1)))) // E
println(process(Pair(7,Pair(1,2)))) // ?
println(process(Pair(7,Pair(2,2)))) // F2
println(process(Pair(8,Pair(1,1)))) // G1.4....
println(process(Pair(8,Pair(0.2,0.2)))) // ?
println(process(Pair(9,new MyPair(10,10)))) // H
```

Companion object

Dealing with static properties/singletons

- given a class/trait `C`, it is an `object C{...}`
- it includes "static properties" of `C`
- it can see private properties of the class and viceversa
- it typically includes:
  - factories
  - `apply`, `unapply` methods
  - other utility methods (e.g., as in `java.util.Collections`)
  - inner classes that would be `static` in Java
  - hidden implementations of the trait

Splitting classes and objects nicely deal with the annoying problem of static properties

Case classes and companion object: takeaway

Phylosophy

Helpers/factories/equals/hashcode/toString: streamlined by case classes and companion objects

Features

- Equals/hascode/toString essentially similar to Java
- == is the equals to be used by anyone!
- case classes give many defaults:
  `apply`, `unapply`, `toString`, `copy`
- all such defaults can be re-programmed at need

Outline

1. Basic OO
2. Uniform OO access in Scala
3. Case classes and companion object
4. Control structures

Control structures, iterations, arrays

Imperative constructs in Scala, philosophy

- they are refurbished in a functional way
- typically translate into standard expressions with no constructs
- in fact, most (i.e., all) constructs could just be libraries in Scala

Imperative constructs in Scala, pragmatics

- straightforward versions of `if` and `while`
- a switch construct by `case` and `match`
- a powerful for-comprehension with `for`

⇒ you can iterate objects with a `foreach` method

Arrays and Variable arguments

- the last argument in parameters can be of type `T*` (understood as `Seq[T]`)
- it indicates a sequence of actual parameters at the call side
- `T*` (i.e., `Seq[T]`) has the `foreach` method..
- arrays `Array[T]` similarly have the `foreach` method

Control structures, some examples

```scala
val b = true
println(if (b) "OK" else "NO") // if is actually an expression
println(if b then "OK" else "NO") // Scala 3 version without parenthesis
println(b match {case true => "OK" case _ => "NO"}) // equivalent version
val s = if b then  // recall that bodies are a special case of expr
  println("OK"); "OK"
else "NO"

val any: AnyVal = if b then 1 else true // least upperbound type!
val i: Int = if b then 1 else throw new Error() // Int vs Nothing

def gcd(x: Int, y: Int): Int = // imperative style gcd
  var a = x
  var b = y
  while (a != b)
    if a > b then a = a - b else b = b - a
  a
println(gcd(10, 18))
```

For comprehension and foreach

```
  // Basic uses of iterative for
  for i <- 1 to 4 do print(i + " ") // guess what 'to' is..
  println() // 1,2,3,4
  for i <- 1 until 4 do print(i + " ")
  println() // 1,2,3
  (4 to 1 by -1).foreach(i => print(i + " ")) // 4,3,2,1
  println()

  // Custom iterations, with a class and its factory in a companion obj
  class ZeroTo(val n: Int):
    def foreach(action: Int => Unit): Unit = for i <- 0 until n do action(i)

  object ZeroTo:
    def apply(n: Int) = new ZeroTo(n)

  for i <- ZeroTo(10) do print(i + " ")
  println()

  // A variant of for yield: a sequence of flatMap and foreach at the end
  for
    i <- 0 until 10
    if i % 3 == 0
    j <- 0 until 2
  do print(s"$i - $j ")
// 0-0 0-1 3-0 3-1 6-0 6-1 9-0 9-1
```

Variable arguments and arrays

```
def sum(args: Int*): Int =
  var s = 0
  for i <- args do s = s + i
  s

println(sum(1,2,3,4,5))

def printall(args: Int*) =
  args foreach (print(_)) // infix notation

printall(1,2,3,4,5)
println

// Arrays are generic classes with proper apply method..
val a: Array[Int] = Array(1,2,3,4,5,6) // a Java-compatible array
a(2)=30
println(s"$a, ${a.apply(1)}, ${a(2)}, ${a.length}")
// ([I@1a407d53,2,30,6)

// equivalent code thanks to Scala's apply, update and varargs
val b: Array[Int] = Array.apply(1,2,3,4,5,6)
// a Java-compatible array
b.update(2,30)
println(s"$b, ${b.apply(1)}, ${b.apply(2)}, ${b.length}")
// ([I@3d8c7aca,2,30,6)
```

Fancy syntax in Scala

One of the assets of Scala

- very flexible syntax
- customising the way libraries can be used
- up to give the impression you have new language constructs
- gives a taste of scripting (e.g. in groovy), but with strong typing

Some mechanisms

- already seen: infix/postfix operators, apply, variable arguments
- default value of arguments
- named arguments
- curly braces instead of single-argument parenthesis
- others to be seen later (called "implicits")

Fancy demo: making Builder often unneeded

```scala
case class Person(name: String, surname: String, year: Int)

def makePerson(name: String, surname: String = "?", year: Int = 1900) = Person(name, surname,
  year)

println(makePerson("bob", "hope")) // skipping args with a default
println(makePerson(name = "bob", surname = "hope")) // using named args
println(makePerson(surname = "hope", name = "bob")) // no order
val p = makePerson(name = "bob", surname = "hope", year = 1920)
println(p)
println(p.copy(year = 1921)) // copy method of case classes
println(makePerson(name = "bob", "hope")) // some name
println(makePerson("bob"))
println{makePerson{"bob"}} // curly braces

@annotation.tailrec // call by-name
def mywhile(condition: => Boolean)(body: =>Unit): Unit =
  if condition then {body; mywhile(condition)(body)}

var n = 5
var res = 1
mywhile( n>=1 ) { // the same as while!!
  res = res * n
  n = n - 1
}
```

Notes on formatting Scala

Some conventions: https://docs.scala-lang.org/style/

- 2 spaces indentation, Java's style for curly brackets
- val/var, methods, classes, and packages as in Java;
- objects like classes, vals in objects start with Uppercase
- getters/setters: `def prop: int`, `def prop_=(n: int)`
- methods without parenthesis only for accessors with no side-effects
- use symbolic methods names for math/logics, only when self-documenting and pure
- because of Scala's roots in the functional languages, it is quite normal for local names to be very short, differently from Java
- short method body in the same line, a longer one in the next line, curly brackets with multiple instructions
- ...

One slide sum-up of OOP in Scala

Key concepts

- traits, classes, fields, methods: as expected
- objects for singleton and static properties
- primary constructor nested with class parameters
- apply/unapply for factories and ad-hoc matching
- case classes for full-fledged records-like objects
- "everything is method call" (to objects)
- fields are twisted with methods
- mono-directional smooth interoperability with Java

Sugar

- imperative programming
- named parameters, `{..}`, optional `()`, infix/prefix calls, symbol names